

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра програмної інженерії

ЗВІТ

з самостійної роботи

з дисципліни «Основи програмної інженерії»

на тему: «Розгортання застосунку на сервері або у хмарі»

Виконав: ст. гр. ПЗПІ-25-6

Коновалов О. О.

Перевірив: Гребенюк В. О.

Харків — 2026

1 МЕТА РОБОТИ

Набути практичних навичок розгортання веб-застосунків на віддаленому сервері. Перетворити програмний модуль системи генерації бібліографічних цитат (з ЛР 03–04) у REST API та забезпечити його безперервну доставку через CI/CD pipeline.

2 ЗАВДАННЯ

1. Взяти відрефакторений модуль з ЛР 03–04 (Citation Generator, TypeScript/Vitest).
2. Створити REST API обгортку на Express із ендпоінтами: GET /health, GET /styles, POST /citations/generate.
3. Підготувати Dockerfile для контейнеризації.
4. Розгорнути застосунок на VPS (Hetzner Cloud) через Docker Compose.
5. Налаштувати CI/CD pipeline (GitHub Actions) для автоматичного деплою при push у main.
6. Додати Swagger UI документацію (/docs).
7. Верифікувати працездатність через публічний URL.

3 ХІД РОБОТИ

3.1 Створення REST API

Для обгортки модуля використано фреймворк Express 5. Сервер приймає запити на порті 3000 (конфігурується через змінну оточення PORT).

Реалізовано три ендпоінти:

- GET /health — перевірка працездатності, повертає JSON з полями status та timestamp;
- GET /styles — список підтримуваних стилів цитування (APA, MLA, Chicago);
- POST /citations/generate — генерація цитати за ідентифікатором (DOI, ISBN або URL) та обраним стилем.

Лістинг 3.1 — Ключовий фрагмент server.ts — обробник POST

/citations/generate

```
app.post('/citations/generate', async (req, res) => {
  const { identifier, style } = req.body;

  if (!identifier || !style) {
    res.status(400).json({ error: 'identifier and style are required' });
    return;
  }

  const validStyles: StyleName[] = ['APA', 'MLA', 'Chicago'];
  if (!validStyles.includes(style)) {
    res.status(400).json({ error: `Invalid style. Use one of: ${validStyles.join(', ')} ` });
    return;
  }

  try {
    const request = new SearchRequest(identifier);
    const citationStyle = new CitationStyle(style);
    const citation = await generator.generate(request, citationStyle);

    res.json({
      citation: citation.getFormattedText(),
      metadata: citation.rawMetadata,
      style: citation.getStyle().name,
    });
  } catch (err) {
    const message = err instanceof Error ? err.message : 'Unknown error';
    res.status(422).json({ error: message });
  }
}
```

3.2 Контейнеризація (Docker)

Створено multi-stage Dockerfile: перший етап (builder) компілює TypeScript у JavaScript, другий — копіює лише скомпільований код та production-залежності.

Лістинг 3.2 — Dockerfile

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.build.json ./
COPY src ./src
RUN npm run build

FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
ENV PORT=3000
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Для оркестрації контейнера використано Docker Compose.

Лістинг 3.3 — docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - PORT=3000
    restart: unless-stopped
```

3.3 Розгортання на VPS

Застосунок розгорнуто на VPS Hetzner Cloud (Ubuntu 20.04, 4GB RAM). Для безпеки створено окремого користувача deploy з обмеженими правами sudo (дозвіл лише на docker-compose).

Кроки розгортання:

1. Встановлено Docker та Docker Compose на сервері.
2. Склоновано репозиторій у /home/deploy/bse-sr-konovarov.
3. Запущено контейнер через docker-compose up -d --build.
4. Верифіковано працездатність через публічний IP.

Публічний URL: <http://78.47.249.61:3000>

3.4 CI/CD Pipeline

Налаштовано GitHub Actions workflow з двома етапами: test (запуск тестів) та deploy (SSH-підключення до VPS, оновлення коду, перезбірка контейнера).

Лістинг 3.4 — Workflow файл .github/workflows/deploy.yml

```
name: CI/CD

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm ci
      - run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
```

```
if: github.ref == 'refs/heads/main'
steps:
  - name: Deploy to VPS
    uses: appleboy/ssh-action@v1
    with:
      host: ${ secrets.VPS_HOST }
      username: ${ secrets.VPS_USER }
      key: ${ secrets.VPS_SSH_KEY }
      script: |
        cd ~/bse-sr-konovalov
        git pull origin main
        sudo docker-compose up -d --build
```

Для безпечної аутентифікації створено виділений SSH-ключ (ED25519), збережений у GitHub Encrypted Secrets. Ключ надає доступ лише користувачу deploy.

3.5 API документація (Swagger UI)

Додано інтерактивну документацію на основі OpenAPI 3.0 специфікації, доступну за адресою /docs. Swagger UI дозволяє тестувати всі ендпоінти безпосередньо у браузері.

4 РЕЗУЛЬТАТИ

Застосунок успішно розгорнуто та доступно за адресою <http://78.47.249.61:3000>.

Результати тестування ендпоінтів:

Таблиця 4.1 — Результати тестування ендпоінтів

Ендпоінт	Метод	Статус	Відповідь
/health	GET	200	{"status":"ok","timestamp":"2026-05-23T10:05:18.234Z"}
/styles	GET	200	{"styles":["APA","MLA","Chicago"]}
/citations/generate	POST	200	Генерація цитати у форматі APA за DOI
/docs	GET	200	Swagger UI інтерфейс

Конфігурація розгортання:

Таблиця 4.2 — Конфігурація розгортання

Параметр	Значення
Платформа	Hetzner Cloud VPS
ОС	Ubuntu 20.04 LTS
Runtime	Docker (node:22-alpine)
Порт	3000
CI/CD	GitHub Actions
Репозиторій	github.com/oleksandrkononov1/bse-sr-kononov

Результати CI/CD: усі запуски pipeline завершилися успішно. Тести (63/63) проходять перед кожним деплоєм.

5 ВИСНОВКИ

У ході виконання самостійної роботи набуто практичних навичок розгортання веб-застосунків. Програмний модуль генерації бібліографічних цитат успішно перетворено на REST API та розгорнуто на віддаленому сервері через Docker. Налаштований CI/CD pipeline забезпечує автоматичну доставку змін при кожному push у main-гілку, що є стандартною практикою у сучасній програмній інженерії.

ДОДАТОК А

Вихідний код server.ts

```
import express from 'express';
import swaggerUi from 'swagger-ui-express';
import { readFileSync } from 'fs';
import { fileURLToPath } from 'url';
import { dirname, join } from 'path';
import { SearchRequest } from './search-request.js';
import { CitationStyle, StyleName } from './citation-style.js';
import { CitationGenerator } from './citation-generator.js';
import { CrossrefProvider } from './crossref-provider.js';
import { OpenLibraryProvider } from './open-library-provider.js';
import { WebScraperProvider } from './web-scraper-provider.js';
import { IdentifierType } from './types.js';
import { MetadataProvider } from './metadata-provider.js';

const __dirname = dirname(fileURLToPath(import.meta.url));
const swaggerDocument = JSON.parse(readFileSync(join(__dirname, 'openapi.json'),
'utf-8'));

const app = express();
app.use(express.json());
app.use((_req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Headers', 'Content-Type');
  res.header('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
  next();
});

app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));
app.get('/', (_req, res) => { res.redirect('/docs'); });

const providers = new Map<IdentifierType, MetadataProvider>([
  [IdentifierType.DOI, new CrossrefProvider()],
  [IdentifierType.ISBN, new OpenLibraryProvider()],
  [IdentifierType.URL, new WebScraperProvider()],
]);
const generator = new CitationGenerator(providers);

app.get('/health', (_req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});

app.get('/styles', (_req, res) => {
  res.json({ styles: ['APA', 'MLA', 'Chicago'] });
});

app.post('/citations/generate', async (req, res) => {
  const { identifier, style } = req.body;

  if (!identifier || !style) {
    res.status(400).json({ error: 'identifier and style are required' });
    return;
  }

  const validStyles: StyleName[] = ['APA', 'MLA', 'Chicago'];
  if (!validStyles.includes(style)) {
    res.status(400).json({ error: `Invalid style. Use one of: ${validStyles.join(',')} ` });
    return;
  }
}
```

```
    try {
      const request = new SearchRequest(identifier);
      const citationStyle = new CitationStyle(style);
      const citation = await generator.generate(request, citationStyle);

      res.json({
        citation: citation.getFormattedText(),
        metadata: citation.rawMetadata,
        style: citation.getStyle().name,
      });
    } catch (err) {
      const message = err instanceof Error ? err.message : 'Unknown error';
      res.status(422).json({ error: message });
    }
  });

  const PORT = parseInt(process.env.PORT || '3000', 10);
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
  });

  export default app;
```

ДОДАТОК Б

Dockerfile

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY tsconfig.build.json ./
COPY src ./src
RUN npm run build

FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
ENV PORT=3000
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

ДОДАТОК В

docker-compose.yml

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - PORT=3000
    restart: unless-stopped
```

ДОДАТОК Г

CI/CD Pipeline (deploy.yml)

```
name: CI/CD

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm ci
      - run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Deploy to VPS
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.VPS_HOST }
          username: ${ secrets.VPS_USER }
          key: ${ secrets.VPS_SSH_KEY }
          script: |
            cd ~/bse-sr-konovalov
            git pull origin main
            sudo docker-compose up -d --build
```